

```
"""
```

```
nonlinear_gr.py - Full nonlinear curvature computation.
```

```
Beyond linearized theory. Computes:
```

```
     $\Gamma^k_{ij}$  - Christoffel symbols from  $g_{ij} = \eta_{ij} + h_{ij}$ 
```

```
     $R^l_{kij}$  - Riemann curvature tensor
```

```
     $R_{ij}$  - Ricci tensor
```

```
     $R$  - Ricci scalar
```

```
     $G_{ij}$  - Full Einstein tensor  $G_{ij} = R_{ij} - (1/2) g_{ij} R$ 
```

```
    Residual:  $||G_{ij} - 8\pi G T_{ij}||_F$ 
```

The metric  $g_{ij}$  at each spatial point  $(x,y)$  is derived from the local field energy density  $\rho(x,y) = |\psi(x,y)|^2$ :

```
     $h_{ij}(x,y) = \Phi(x,y) \cdot \delta_{ij}$ 
```

```
     $\nabla^2 \Phi = -4\pi G \rho$  (Poisson equation for gravitational potential)
```

```
     $g_{ij} = (1 + 2\Phi/c^2) \delta_{ij}$  (isotropic weak field metric)
```

This is the Newtonian limit of GR but computed nonlinearly -  $\Phi$  is solved exactly for the given  $\rho$ , then Christoffel symbols are computed without assuming  $|h| \ll 1$ .

For strong fields ( $|\Phi| \sim 1$ ), the nonlinear terms in  $\Gamma$  become significant and the result departs from the linearized Einstein tensor.

```
"""
```

```
import numpy as np
```

```
import math
```

```
from dataclasses import dataclass
```

```
from typing import Optional
```

```
@dataclass
```

```
class CurvatureResult:
```

```
    """Full curvature decomposition at one field state."""
```

```
    # Metric
```

```
    g: np.ndarray          # (2,2,N,N) full metric
```

```
    g_inv: np.ndarray      # (2,2,N,N) inverse metric
```

```
    Phi: np.ndarray        # (N,N) gravitational potential
```

```
    # Curvature
```

```
    Gamma: np.ndarray      # (2,2,2,N,N) Christoffel symbols  $\Gamma^k_{ij}$ 
```

```
    Ricci: np.ndarray      # (2,2,N,N) Ricci tensor  $R_{ij}$ 
```

```
    Ricci_scalar: np.ndarray # (N,N)  $R$ 
```

```

# Einstein tensor
G_full: np.ndarray      # (2,2,N,N) full nonlinear G_ij
G_linear: np.ndarray     # (2,2,N,N) linearized G_ij^(1) for comparison

# Stress tensor
T: np.ndarray           # (2,2,N,N) MSR stress tensor

# Residuals
nonlinear_residual: float # ||G_full - 8πG T|| / ||G_full||
linear_residual: float    # ||G_linear - 8πG T|| / ||G_linear||
nonlinearity: float       # ||G_full - G_linear|| / ||G_full||

# Spatial means
G_mean: np.ndarray
T_mean: np.ndarray
kappa: float

class NonlinearEinsteinSolver:
    """
    Computes the full nonlinear Einstein tensor from the field  $\psi$ .

    Usage::
        solver = NonlinearEinsteinSolver(cfg)
        result = solver.compute(psi, psi_hat)
        print(f"Nonlinearity: {result.nonlinearity:.4f}")
        print(f"Nonlinear residual: {result.nonlinear_residual:.4f}")
        print(f"Linear residual:      {result.linear_residual:.4f}")
    """

    def __init__(self, cfg, G_newton: float = 1.0, c: float = 1.0):
        self.N = cfg.grid.N
        self.dx = 1.0 / cfg.grid.N
        self.G = G_newton
        self.c = c

        kx = 2*math.pi*np.fft.fftfreq(self.N, d=self.dx)
        KX, KY = np.meshgrid(kx, kx)
        self.k2 = KX**2 + KY**2
        self.k2[0,0] = 1.0

    def _grad(self, f: np.ndarray, axis: int) -> np.ndarray:
        return np.gradient(f, self.dx, axis=axis)

    def _poisson(self, rho: np.ndarray) -> np.ndarray:
        """Solve  $\nabla^2\Phi = -4\pi G \rho$  exactly via FFT."""

```

```

rho_k = np.fft.fft2(rho)
Phi_k = 4 * math.pi * self.G * rho_k / self.k2
Phi_k[0,0] = 0.0
return np.fft.ifft2(Phi_k).real

def _build_metric(self, Phi: np.ndarray) -> tuple:
    """
    g_ij = (1 + 2Φ/c²) δ_ij   (isotropic conformal metric)
    g^ij = (1 - 2Φ/c²) δ_ij   (inverse)
    """
    N = self.N
    factor = 1.0 + 2*Phi/self.c**2
    g      = np.zeros((2,2,N,N))
    g_inv  = np.zeros((2,2,N,N))
    g[0,0] = factor
    g[1,1] = factor
    g_inv[0,0] = 1.0/factor
    g_inv[1,1] = 1.0/factor
    return g, g_inv

def _christoffel(self, g: np.ndarray, g_inv: np.ndarray) -> np.ndarray:
    """
    Γ^k_ij = (1/2) g^kl (∂_i g_lj + ∂_j g_li - ∂_l g_ij)
    Returns (2,2,2,N,N): Gamma[k,i,j]
    """
    N = self.N
    Gamma = np.zeros((2,2,2,N,N))

    # ∂_m g_ij   (shape 2,2,2,N,N: dg[m,i,j])
    dg = np.zeros((2,2,2,N,N)) # dg[m,i,j] = ∂_m g_ij
    for mm in range(2):
        for ii in range(2):
            for jj in range(2):
                dg[mm,ii,jj] = self._grad(g[ii,jj], mm)

    for k in range(2):
        for i in range(2):
            for j in range(2):
                s = np.zeros((N,N))
                for l in range(2):
                    s += g_inv[k,l] * (dg[i,l,j] + dg[j,l,i] - dg[l,i,j]) #
                Gamma[k,i,j] = 0.5 * s

    return Gamma

def _ricci(self, Gamma: np.ndarray) -> np.ndarray:
    """

```

```

R_ij =  $\partial_k \Gamma^k_{ij} - \partial_j \Gamma^k_{ik} + \Gamma^k_{kl} \Gamma^l_{ij} - \Gamma^k_{jl} \Gamma^l_{ik}$ 
Returns (2,2,N,N)
"""
N = self.N
R = np.zeros((2,2,N,N))
for i in range(2):
    for j in range(2):
        for k in range(2):
            #  $\partial_k \Gamma^k_{ij}$ 
            R[i,j] += self._grad(Gamma[k,i,j], k)
            #  $-\partial_j \Gamma^k_{ik}$ 
            R[i,j] -= self._grad(Gamma[k,i,k], j)
            #  $\Gamma^k_{kl} \Gamma^l_{ij} - \Gamma^k_{jl} \Gamma^l_{ik}$ 
            for l in range(2):
                R[i,j] += Gamma[k,k,l]*Gamma[l,i,j]
                R[i,j] -= Gamma[k,j,l]*Gamma[l,i,k]
return R

def _linear_einstein(self, h_ij: np.ndarray) -> np.ndarray:
    """Linearized  $G^{(1)}[h]$  for comparison."""
    N = self.N
    G = np.zeros((2,2,N,N))
    h_tr = h_ij[0,0] + h_ij[1,1]

    def lap(f):
        return (np.roll(f,1,0)+np.roll(f,-1,0)+
                np.roll(f,1,1)+np.roll(f,-1,1)-4*f)/self.dx**2

    div = [sum(self._grad(h_ij[mu,nu],mu) for mu in range(2))
            for nu in range(2)]
    div2 = sum(self._grad(self._grad(h_ij[mu,nu],mu),nu)
                for mu in range(2) for nu in range(2))

    for i in range(2):
        for j in range(2):
            eta = 1.0 if i==j else 0.0
            G[i,j] = 0.5*(
                self._grad(div[j],i) + self._grad(div[i],j) -
                lap(h_ij[i,j]) -
                self._grad(self._grad(h_tr,i),j) -
                eta*(div2 - lap(h_tr))
            )
    return G

def compute(self, psi: np.ndarray, psi_hat: np.ndarray) -> CurvatureResult:
    """
    Full nonlinear Einstein computation from field state.

```

```

"""
N = self.N

# 1. Source: energy density  $\rho = |\psi|^2$ 
rho = np.abs(psi)**2

# 2. Gravitational potential from Poisson
Phi = self._poisson(rho)

# 3. Metric  $g_{ij}$  and inverse
g, g_inv = self._build_metric(Phi)

# 4. Christoffel symbols (nonlinear)
Gamma = self._christoffel(g, g_inv)

# 5. Ricci tensor (nonlinear)
R_ij = self._ricci(Gamma)

# 6. Ricci scalar  $R = g^{ij} R_{ij}$ 
R_scalar = sum(g_inv[i,i] * R_ij[i,i] for i in range(2))

# 7. Full nonlinear Einstein tensor
G_full = np.zeros((2,2,N,N))
for i in range(2):
    for j in range(2):
        G_full[i,j] = R_ij[i,j] - 0.5 * g[i,j] * R_scalar

# 8. Linearized Einstein tensor for comparison
h_ij = g - np.array([[np.ones((N,N)), np.zeros((N,N))],
                      [np.zeros((N,N)), np.ones((N,N))]])
G_lin = self._linear_einstein(h_ij)

# 9. MSR stress tensor  $T_{\mu\nu}$ 
d = [self._grad(psi.real, i) for i in range(2)]
dh = [self._grad(psi_hat.real, i) for i in range(2)]
T = np.zeros((2,2,N,N))
for i in range(2):
    for j in range(2):
        T[i,j] = dh[i]*d[j]+dh[j]*d[i]
tr = sum(dh[i]*d[i] for i in range(2))
T[0,0] -= tr; T[1,1] -= tr

# 10. Residuals
coeff = 8 * math.pi * self.G
G_f_mean = G_full.mean(axis=(-2,-1))
G_l_mean = G_lin.mean(axis=(-2,-1))
T_mean = T.mean(axis=(-2,-1))

```

```

Gf_n = np.linalg.norm(G_f_mean)+1e-15
Gl_n = np.linalg.norm(G_l_mean)+1e-15
T_n  = np.linalg.norm(T_mean)+1e-15

kappa_nl = float(np.sum(T_mean*G_f_mean)/Gf_n**2)
kappa_l  = float(np.sum(T_mean*G_l_mean)/Gl_n**2)

nl_res  = float(np.linalg.norm(T_mean - kappa_nl*G_f_mean)/T_n)
lin_res = float(np.linalg.norm(T_mean - kappa_l*G_l_mean)/T_n)
nonlin  = float(np.linalg.norm(G_f_mean - G_l_mean)/Gf_n)

return CurvatureResult(
    g=g, g_inv=g_inv, Phi=Phi,
    Gamma=Gamma,
    Ricci=R_ij, Ricci_scalar=R_scalar,
    G_full=G_full, G_linear=G_lin,
    T=T,
    nonlinear_residual=nl_res,
    linear_residual=lin_res,
    nonlinearity=nonlin,
    G_mean=G_f_mean, T_mean=T_mean,
    kappa=kappa_nl,
)

```